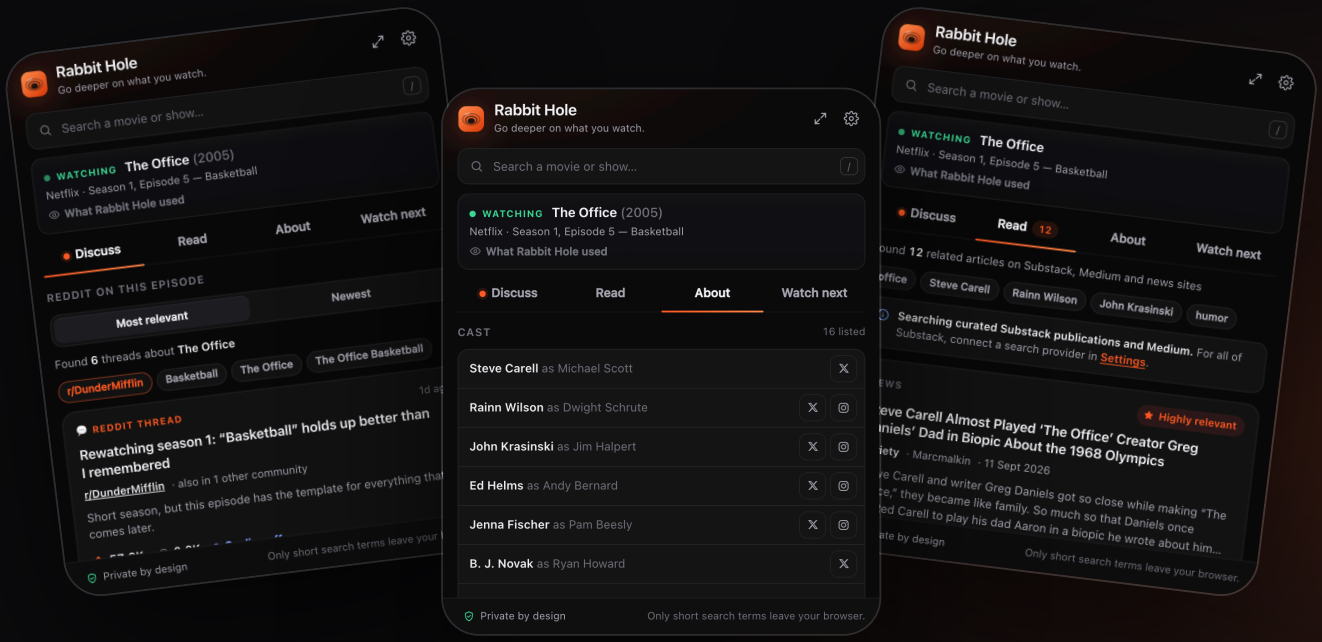
PROJECT REPORT

Rabbit Hole

Go deeper on what you watch — Discuss · Read · About · Watch next, for any Netflix movie or show.

Chrome · Manifest V3 | TypeScript · React 19 | Netflix × Reddit × Wikidata | News & Substack | Privacy-first



Harsh Giri · Version 1.0.0

11 October 2026

Contents

	Executive summary	3
CH 1	**Problem, goals and users**	4
CH 2	**Product tour**	7
CH 3	**UX and visual design**	12
CH 4	**System design**	14
CH 5	**Flowcharts**	18
CH 6	**Privacy and security**	25
CH 7	**Testing and quality assurance**	26
CH 8	**Usability evaluation plan**	28
CH 9	**Limitations, risks and future work**	30
APP	**Appendices**	31

Companion deliverables (in `docs/`): `Rabbit-Hole-Heuristic-Evaluation-Form.pdf` — fillable usability & heuristic evaluation pack · `Rabbit-Hole-Heuristic-Evaluation-Results.pdf` — a two-page expert heuristic review of the round-1 build · `flowcharts/` — project flowchart, system architecture, Figma-ready UI flow (SVG/PNG/Mermaid) · `screens/` — every UI state as PNG.

Executive summary

Rabbit Hole is a Chrome extension (Manifest V3) for people who watch **Netflix**. It works out exactly which movie or show is playing — or lets you search for any title — and gives four views of it: what **Reddit** is saying (**Discuss**), thoughtful writing and press coverage (**Read**), the cast and where to find reviews (**About**), and what to watch next (**Watch next**).

263
automated tests in 21 files, all passing

31/31
real-Chrome end-to-end checks (Chrome 155)

12/12
real Netflix titles resolved to the right Wikidata item

22
news outlets via public RSS (+ 61 Substack publications)

What was built

- **One exact title.** “The Office (U.S.)” on Netflix is matched to the American sitcom on Wikidata — not the British original — using the catalogue tag, kind, year and cast. Off Netflix, a title search (“Search a movie or show…”) does the same for anything.
- **Discuss** — Reddit threads about that title: the show’s own subreddit first, then TV/film communities, then all of Reddit, keeping only threads that are clearly about the show. Sort by *Most relevant* or *Newest*.
- **Read** — Substack, Medium and (opt-in) 22 news outlets such as The Indian Express, Hindustan Times, The New York Times, The Guardian and Variety, ranked with a transparent 0–100 relevance score and a plain-language *“Why this is relevant”*. Users choose which sources they prefer or never want to see.
- **About** and **Watch next** — cast in billing order with their X and Instagram, links to IMDb, Letterboxd and Rotten Tomatoes, and titles that share the cast or creator. With the optional backend: ratings, viewer reviews, similar titles and whether a title is **on Netflix India**.
- **Privacy as a feature** — only Netflix is ever read, only when the extension is opened; API keys live only on the backend; caches are RAM-only; nothing is tracked.

How it was verified

Unit and integration tests cover title resolution, the Discuss filter, ranking, news parsing, caching, the backend and failure handling; a release gate proves development keys cannot reach a production bundle; and an end-to-end suite loads the *built* extension in a real Chrome and drives the real popup against the real service worker — including live Wikidata, Substack and Medium. The design went through two rounds of feedback (§1.6); a usability-evaluation pack accompanies this report.

> **Scope note**
>
> Reddit blocks the build machine’s network, so Discuss is verified with Reddit’s live RSS from Node, captured feeds and mocked responses. The backend’s TMDB, OMDb, NYT and Guardian integrations are tested against their documented response formats — no keys were available for a live call. Netflix detection is verified against fixtures that mirror its markup. See §9.1.

CHAPTER 1

Problem, goals and users

1.1 Problem statement

Curiosity strikes while watching: a character, an actor, a scene, a question. Following it means leaving Netflix, guessing a search query and sorting through results built for clicks. The conversation (Reddit), considered writing (essays and reviews), the press, and the facts (cast, ratings, what else to watch) all live in different places — and generic searches mix them up with things that merely share the name ("The Office" brings up office politics).

1.2 Goals and non-goals

#	Goal	Measure of success
G1	Know exactly which movie or show is playing on Netflix, or let the user pick one	12/12 real Netflix titles resolve correctly; "don't guess" when unsure (tested)
G2	Show Reddit discussion about *that* title only	Off-topic threads (office politics, "officer") dropped on live data; own subreddit first
G3	Recommend relevant reading from many sources and explain every pick	Relevance 0–100 with labels; "Why this is relevant"; Substack · Medium · news
G4	Give the cast, reviews and what to watch next	Cast in billing order with socials; IMDb / Letterboxd / RT links; same-cast picks
G5	Let users choose their sources	Prefer / Normal / Hide per source; only-my-sources; English only
G6	Privacy first: read only Netflix, no history, keys off the client	Permission audit; real-Chrome check that other sites are unreadable; release gate
G7	Keep working when sources are blocked, limited or offline	Provider chains, circuit breaker, stale-if-error, fallbacks for Wikidata's query service

Non-goals: scraping websites; bypassing Netflix DRM or authentication; reading any site except Netflix; collecting browsing history; guessing facts that no source provided.

1.3 Target users (illustrative personas)

The curious viewer

Aanya, 20 · undergraduate

Watches series on Netflix and wants the conversation and the "why" behind them — fan theories, reviews, interviews — without a research detour.

Needs: instant, on-topic results, low effort

The film buff

Rahul, 34 · product manager

Checks Letterboxd and IMDb before committing, follows actors, and wants to know what else is on Netflix India from the same people.

Needs: ratings, cast, what to watch next

The privacy-conscious reader

Meera, 41 · journalist

Reluctant to install extensions that "read every page". Will adopt only if she can see and control exactly what is read and sent — and pick her own news sources.

Needs: transparency, control, source choice

1.4 The core journey

1. **Watching on Netflix (or search a title)** → 2. **Rabbit Hole knows exactly which title** → 3. **What Reddit says about it** → 4. **What's been written about it** → 5. **Who's in it · what to watch next**

1.5 Requirements

ID	Requirement	Status	Evidence
FR1	Detect a Netflix episode or movie; resolve it to one exact title	Done	netflixExtractor, titleService tests; e2e
FR2	Title search for any movie or show (Wikidata)	Done	titleService tests; e2e (type → pick)
FR3	Discuss: Reddit threads about the title; relevant / newest sort	Done	discuss tests; live RSS check
FR4	Read: Substack, Medium, news outlets, press archives; relevance + explanation	Done	relevance, medium, news, insights tests
FR5	Your sources: prefer / hide / only preferred / English only	Done	news tests
FR6	About: cast with X / Instagram; IMDb, Letterboxd, RT, Wikipedia links	Done	titleService tests; e2e (live Wikidata)
FR7	Watch next: same cast / creator; filters; similar + Netflix India with backend	Done	e2e (live Wikidata); insights tests
FR8	Backend: TMDB, OMDb, NYT, Guardian with keys server-side	Done — not run live	insights tests (documented formats)
NFR1	Cache aggressively; de-duplicate; cancel stale requests	Done	cache tests
NFR2	Strict CSP; validated, sanitised external data	Done	manifest + release gate + e2e
NFR3	Minimal permissions; news sites only when allowed	Done	manifest tests; e2e
NFR4	Keyboard and screen-reader support	Done — audit pending	Usability form, Method F

1.6 Iterations from feedback

The product changed substantially across two rounds of reviewer feedback. Each point was discussed before implementation; the decisions are summarised below.

Round	Feedback	Response
1	"Trending now" is not needed	Removed everywhere: no generic trending feed; everything shown is about something the user is watching or asked for
1	No Reddit results for "The Office" S1E5	Discuss searches all time, by show and episode name

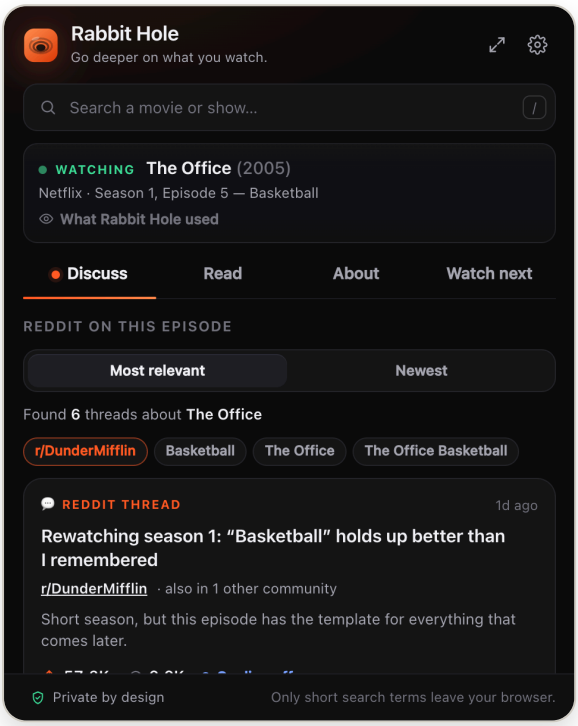
Round	Feedback	Response
1	Related reading too narrow (Substack only)	Medium tag feeds added (keyless)
2	Focus on movies and series on Netflix only	YouTube, Reddit and generic page reading removed; only Netflix is read
2	“The Office” returned office-politics subreddits	Show’s own subreddit first; TV/film communities; an “is it about the show?” filter (§4.4)
2	Cast social links; reviews from IMDb / Letterboxd	About tab: cast with X / Instagram; review links; ratings and reviews via the backend
2	Similar titles, same cast or genre; IMDb-style filters	Watch next: same cast / creator (Wikidata), TMDB similar titles, type · decade · sort filters
2	More news publications; let users pick sources	22 outlets (India, US, UK) via RSS; Your sources: prefer / hide / only preferred
2	Netflix availability for India	On Netflix India from TMDB watch providers (JustWatch), via the backend
2b	Discuss needs a relevancy / recency filter	Most relevant ⇄ Newest switch, remembered; Newest asks Reddit for its newest matches

CHAPTER 2

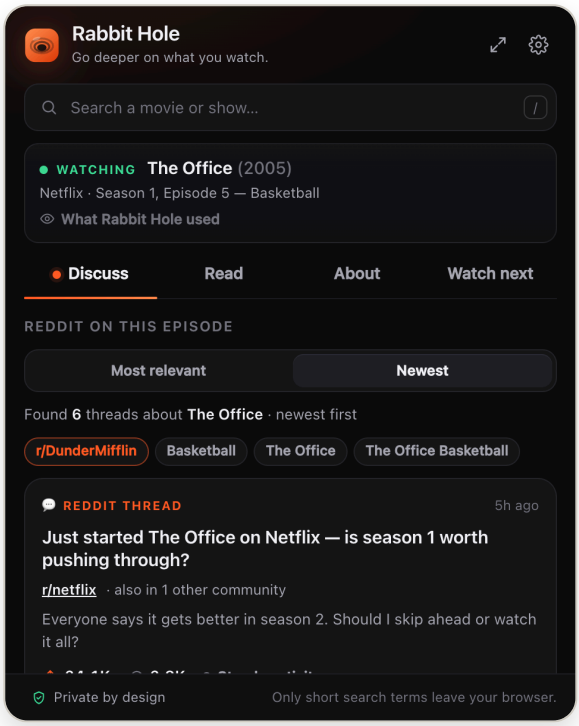
Product tour

Every screen below is a capture of the built interface (480 × 600 px, Chrome's maximum popup height). Reddit threads are **illustrative sample data**; news articles are real (captured from the outlets' feeds on 10 October 2026); Substack articles come from real public RSS; title facts are live Wikidata. Screens marked *backend* use placeholder ratings and availability.

2.1 Discuss — Reddit, about this title only

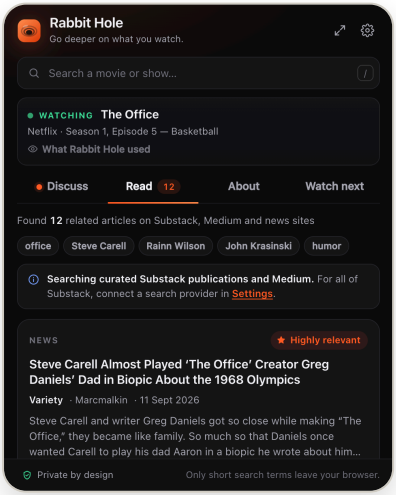


Watching The Office S1E5. The subject bar shows exactly what was identified. Discuss searched the show's own subreddit (r/DunderMifflin chip), the episode and the show; an off-topic r/antiwork thread in the sample was dropped.

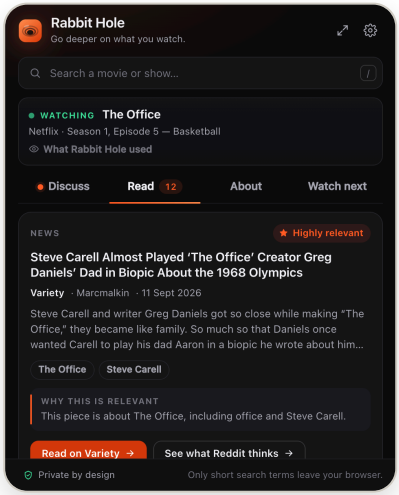


Most relevant ⇄ Newest. "Newest" asks Reddit for its newest matches — not a re-shuffle — and keeps the same "about the show" filter. The choice is remembered.

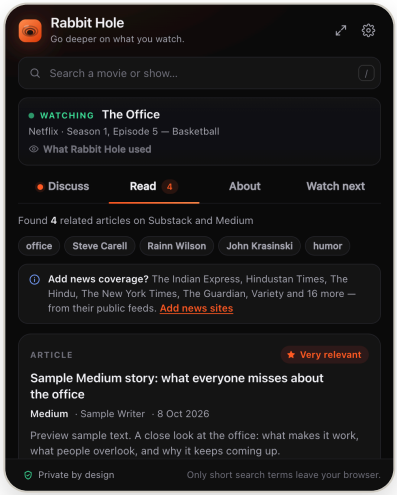
2.2 Read — writing and press coverage



Read with news sites on: Variety, Deadline, Decider… alongside Substack and Medium. Topics searched are shown.

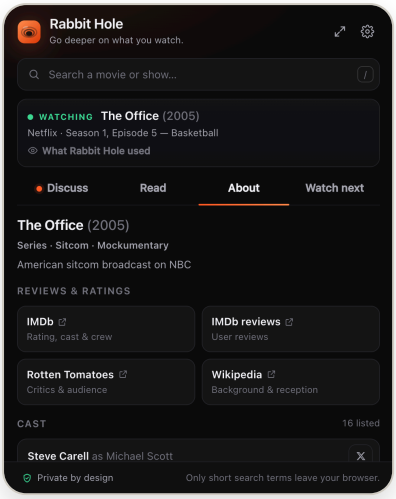


Card: relevance label, outlet · author · date, excerpt and a grounded *Why this is relevant*.

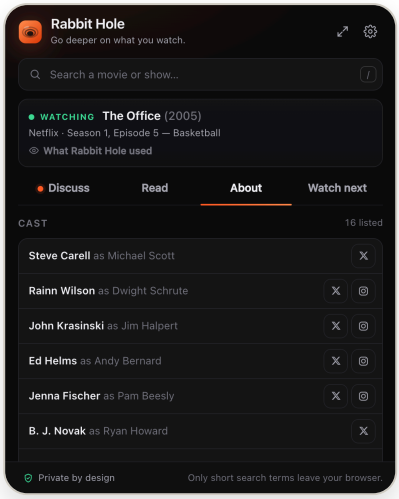


News is opt-in: a one-time offer; Chrome asks for those sites only then.

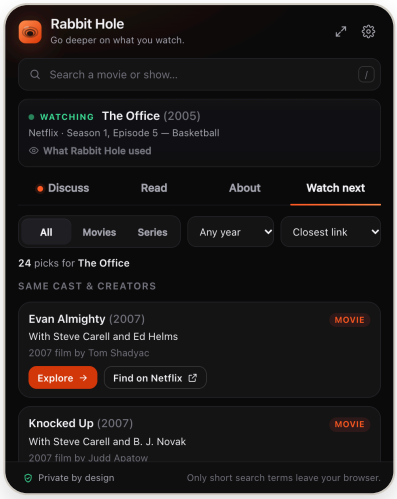
2.3 About and Watch next



About: facts and review links (IMDb, Letterboxd for films, Rotten Tomatoes, Wikipedia) built from Wikidata ids.

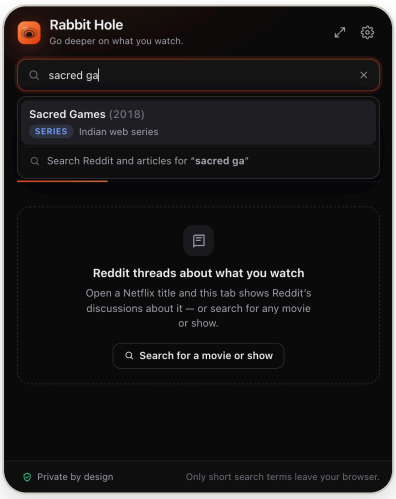


Cast in billing order with characters and their X / Instagram accounts.

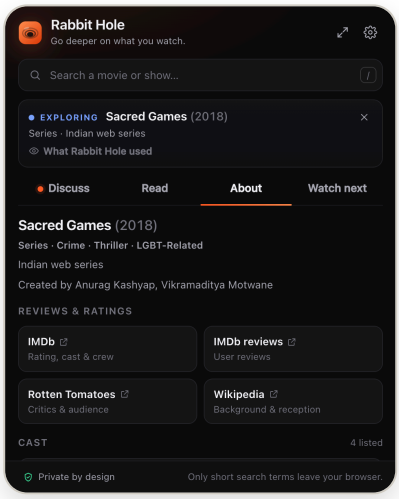


Watch next: titles with the same cast or creator, each with the real link; filters for type, decade and sort.

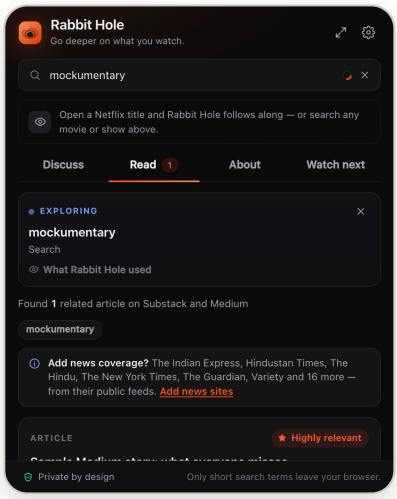
2.4 Title search — any movie or show



Search a movie or show: Wikidata movies and series with year and kind; ↑ ↓ Enter. The last row searches the words themselves.

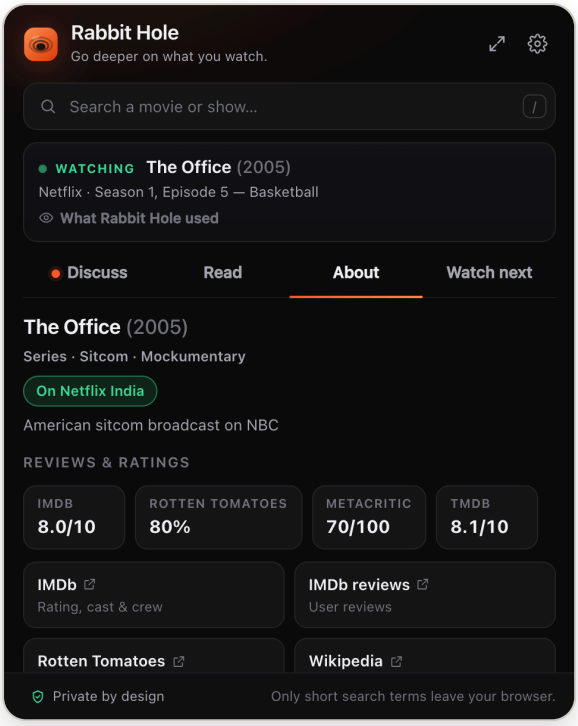


A picked title (“Exploring Sacred Games”): every tab follows it; × returns to what’s playing.

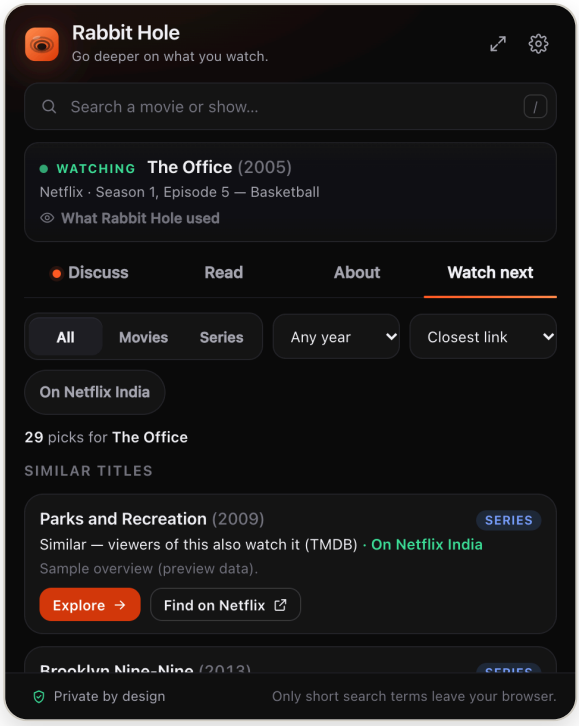


Free-text search (“mockumentary”) drives Discuss and Read only.

2.5 With the backend (placeholder data)

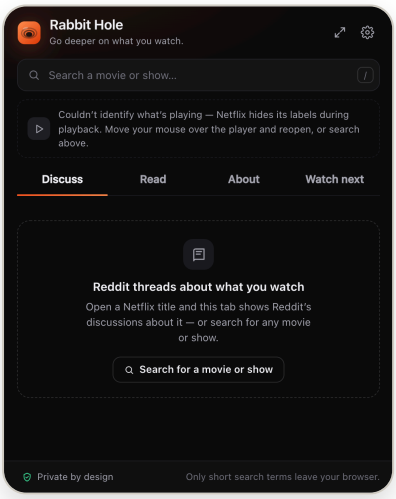


About + backend: IMDb, Rotten Tomatoes, Metacritic and TMDB ratings, viewer reviews and an *On Netflix India* badge. **The numbers in this capture are placeholders** from the preview's sample backend.

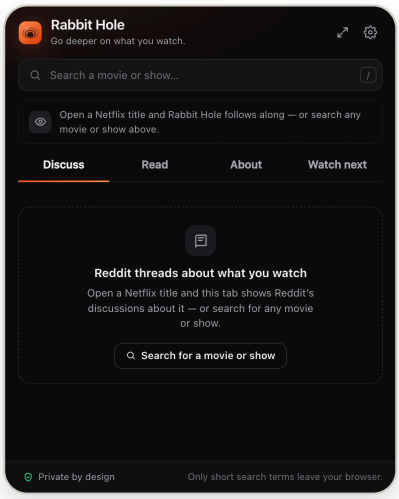


Watch next + backend: TMDB similar titles, each checked for Netflix India, with an *On Netflix India* filter. **Availability here is placeholder data.**

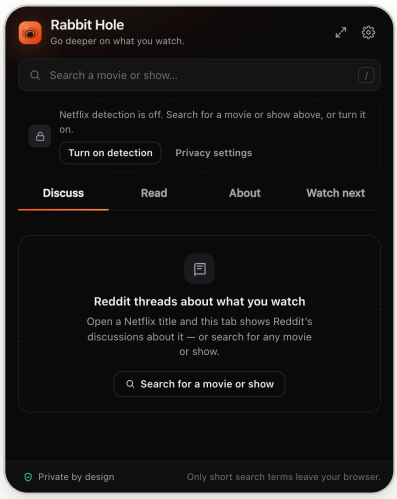
2.6 Other entry states, errors and appearance



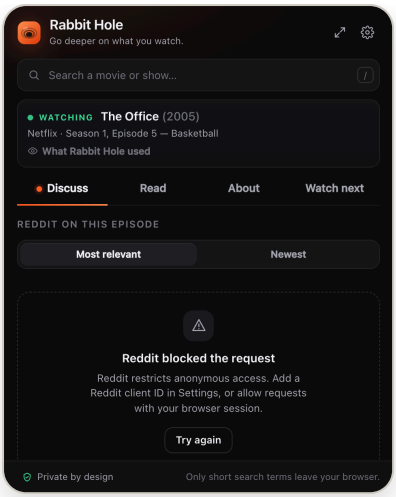
Netflix hides its labels: says why and how to fix it — or search. Never guesses.



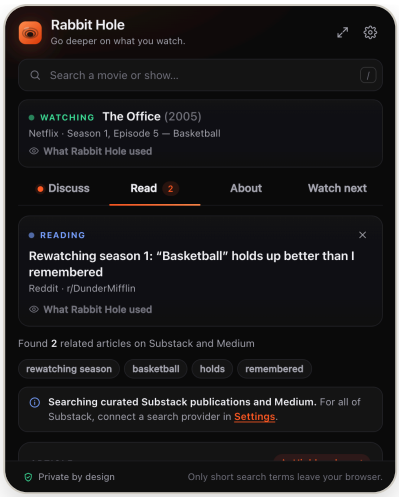
Any other site: nothing is read; the extension cannot even see the URL.



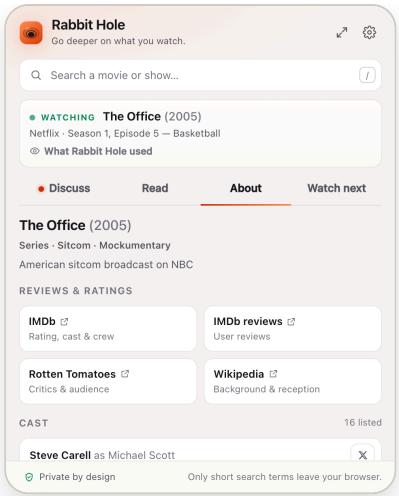
Detection off: the privacy choice is respected; turn it on or search.



Error state: plain cause, one clear action; stale data is shown (flagged) when available.

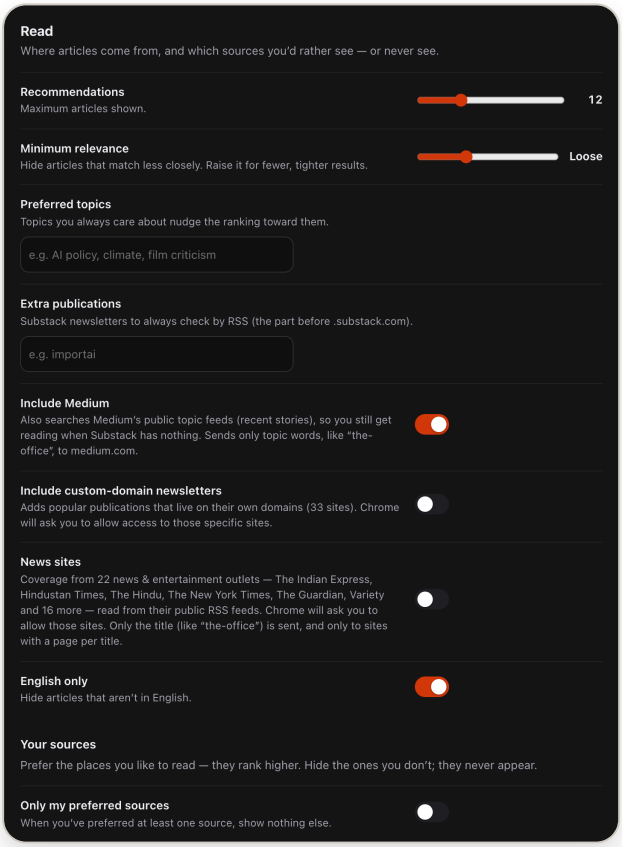


Bridge: "Find deeper reading →" on a thread turns its headline into a Read search.

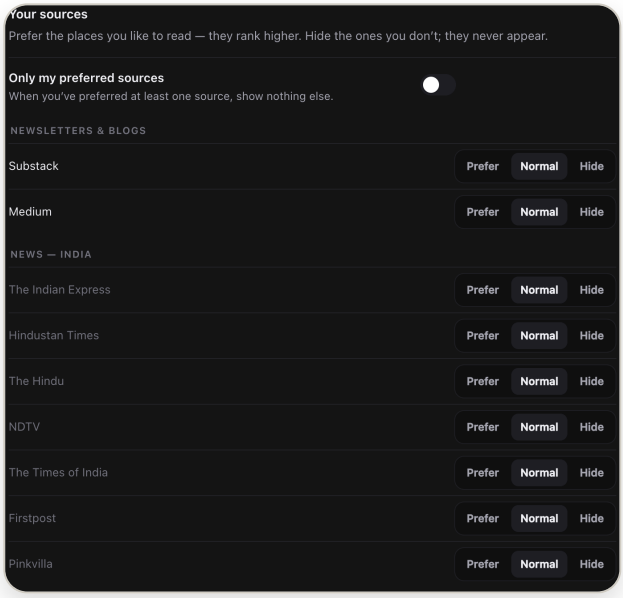


Light theme: dark by default; Light and System available.

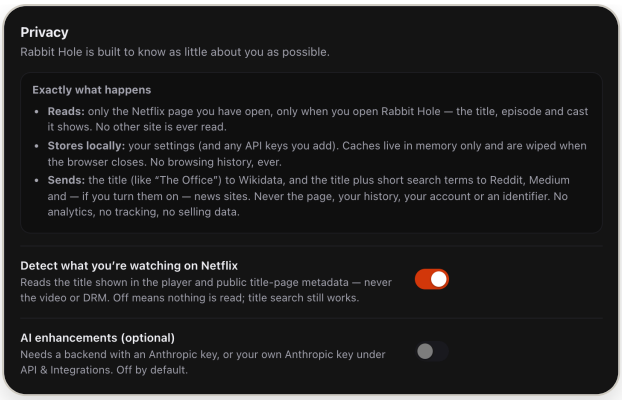
2.7 Settings and the full dashboard



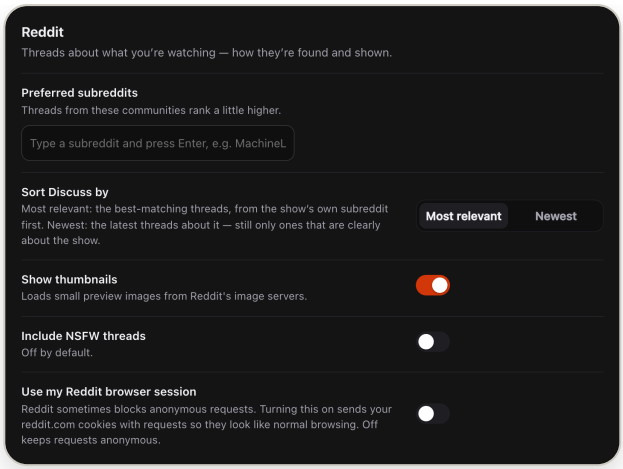
Settings → Read: news sites (asks Chrome for those sites), English only, Medium, relevance threshold.



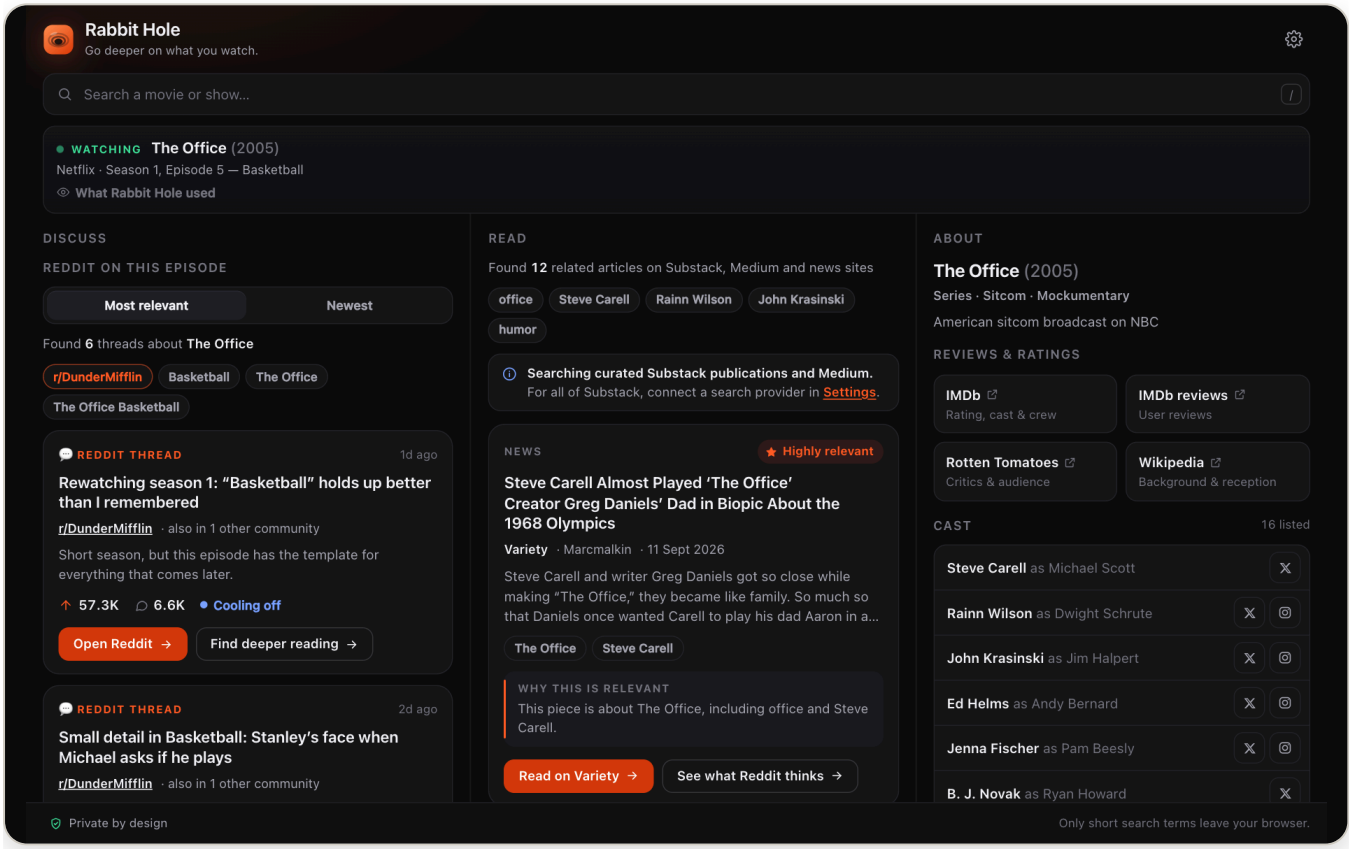
Your sources: Prefer · Normal · Hide for Substack, Medium and every outlet, grouped India / US / UK; *Only my preferred sources*.



Privacy spells out what is read, stored and sent; one Netflix detection switch; AI is opt-in.



Reddit: Sort Discuss by (relevant / newest), preferred subreddits, browser-session option.



Full dashboard (⤢): the same app in a tab — three columns when wide: Discuss | Read | About + Watch next. It exists because Chrome caps popups at 600 px tall.

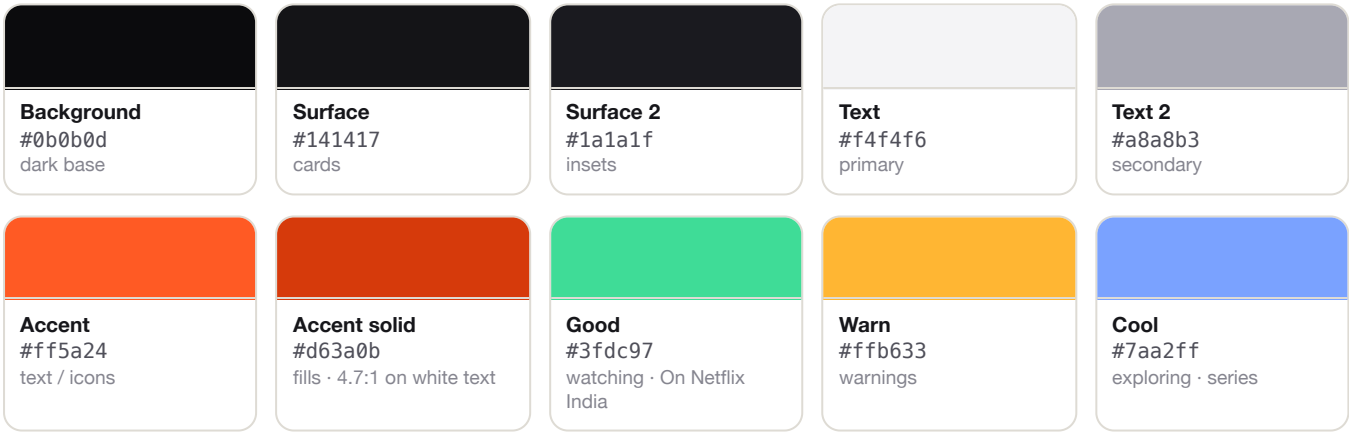
CHAPTER 3

UX and visual design

3.1 Design principles

1. **Say what was understood.** The subject bar (“Watching — The Office (2005) · Netflix · Season 1, Episode 5”) is always visible above the tabs, so every tab is visibly about the same thing — and users can correct it with the search.
2. **One subject, four questions.** Discuss · Read · About · Watch next answer “what are people saying, what’s been written, who’s in it, what next” about the same title.
3. **Never blank, never silent.** Skeleton loaders, a live status line and an explanation for every empty or failed state.
4. **Honest by default.** Missing data stays missing; when a title can’t be pinned down the tab says so rather than guessing; every Watch next pick states its real connection; relevance is a label, not a fake percentage.
5. **User in control.** Most relevant ⇄ Newest, Prefer / Hide sources, × back to Netflix, Esc to close, per-feature permissions, delete-everything.

3.2 Visual language



Aspect	Decision
Typography	System UI sans stack — no remote fonts (privacy, CSP, speed); 13.5 px base; tabular numerals for counts and ratings
Layout	480 × 600 popup: header, title search, subject bar, four tabs, scrolling panel, pinned privacy footer. Dashboard: three columns ≥ 980 px
Components	Combobox (title search), subject bar, segmented controls (sort, type), rating chips, link tiles, cast rows with social buttons, kind pills (Movie / Series)
Shape & depth	10–18 px radii, 1 px hairline borders, no heavy shadows; hover raises border contrast
Motion	150–280 ms transitions; cards “rise” in; pulsing live dot; all disabled under `prefers-reduced-motion`
Theming	CSS custom properties; dark default; Light and System; theme applied before first paint (no flash)

3.3 Accessibility measures built in

Area	Implementation
Keyboard	“/” focuses search; ↑ ↓ Enter pick a title, Esc closes (twice: clears); ←/→/Home/End move between tabs (roving tabindex); every control is a real button or link
Screen readers	WAI-ARIA 1.2 combobox with `aria-activedescendant` and a live result count; tablist/tabpanel roles; radio groups for sort and filters; labelled social links (“Steve Carell on X (@SteveCarell)”)
Focus	Visible 2 px accent focus ring on all interactive elements
Colour	Filled buttons use #d63a0b (4.7:1 with white); availability and kind are conveyed by text, never colour alone
Motion	Animations disabled for reduced-motion users
Target size	Buttons 28–38 px tall; social buttons 30 × 30 (WCAG 2.2 minimum 24 px)

A formal WCAG 2.2 AA audit is part of the evaluation pack (Method F); the above are design intentions, not a certification.

CHAPTER 4

System design

4.1 Architecture at a glance

Three runtime contexts. A single **content script**, on Netflix only, does nothing until asked, then reads the player label and the title page's metadata once. **Extension pages** (popup, dashboard, settings) are a React app that never touches the network. The **background service worker** owns everything else — Wikidata, Reddit, feeds, the backend, caching, ranking — and talks to the pages over a typed, cancellable RPC port. An optional **backend** holds every API key. The full diagram is in §4.7.

4.2 Identifying the exact title (Wikidata)

Step	How
Search	`wbsearchentities` (labels and aliases), then one SPARQL query keeps only items whose *instance of* is a film or series class and adds the year. If the query service is down, Wikidata's own descriptions ("2022 film directed by …") classify instead
Resolve a Netflix page	Score candidates by exact name (+3), kind — series vs movie (+3), Netflix's catalogue tag ("(U.S.)" → "American …", +2), year (+2) and Wikidata's rank; for remaining ties, overlap with the cast Netflix lists (+2 each). Below a threshold: **no match** rather than a guess
Facts	`wbgetentities` : cast in statement (billing) order with characters, director / creator, genres, own subreddit, IMDb / Letterboxd / Rotten Tomatoes / TMDB ids, Wikipedia; SPARQL adds current X / Instagram usernames
Links	Built only from ids that pass strict patterns (`tt\d+` , slugs, username rules); Letterboxd via its `/imdb/<id>/` redirect when it has no slug
Verified live	12 real Netflix titles — The Office (U.S.), Black Mirror, Glass Onion, Sacred Games, RRR, Money Heist, Stranger Things, Delhi Crime, Squid Game, 3 Body Problem, Wednesday, Heeramandi — all resolved correctly

4.3 The context pipeline

Stage	Module	What it does
1 · Platform	`context/platform.ts`	Pure URL logic: Netflix watch / title pages only; every other site is "unknown" and never read
2 · Metadata	`context/extractors/netflix.ts`	Player label ("S1:E5 · Basketball"), Media Session, title-page JSON-LD: title, episode, genres, cast
3 · Topics & entities	`context/topics.ts`	Clean title (strips "(U.S.)", keeps it as the catalogue tag) → phrases → entities → genre themes
4 · Exact title	`titleService.ts`	Wikidata resolve (§4.2) — or the user's pick from the title search
5 · Queries	`context/queries.ts`	3–5 focused queries for Read; Discuss builds its own plan (§4.4)
6 · Four tabs	services	Discuss · Read · About · Watch next, each cached and cancellable

4.4 Algorithms

Discuss — is this thread about the show? (`reddit/aboutTitle.ts`)

```
plan: own subreddit (episode, or top / newest threads) → TV or film subreddits → all
of Reddit (≤ 4 searches, all time)
```

- **Keep** if in the show's own subreddit; otherwise the title must appear as a phrase *and* the thread must be in a TV / film community, write the title as a name ("The Office", not "the office"), mention a cast or character name, or talk like a viewer ("episode", "season", "rewatch"…)
- **Work, career, office-furniture and Microsoft-Office subreddits** need a cast or character name to get in at all
- **Rank** = (0.4 + match) × where (own subreddit 1.5, TV/film 1.2) × $(1 + \log_{10}(1 + \text{momentum}) + 0.25 \cdot \log_{10}(1 + \text{upvotes}))$; or **newest first** when the user chooses it
- On live data, of 22 all-of-Reddit results for "The Office" it kept the r/DunderMifflin, r/theoffice and r/television threads and dropped police-"officer" videos, return-to-office complaints and Oval Office news

Read — article relevance (0–100)

```
relevance = 100 × (0.45·semantic + 0.20·keyword + 0.15·entity + 0.10·recency +
0.10·sourceQuality)
```

- **semantic**: concept-weighted cosine similarity over stemmed terms; common stems down-weighted like a cheap IDF · **recency**: $e^{-\text{age}/150\ \text{days}}$, unknown dates neutral — never guessed
- **Labels**: Highly relevant ≥ 70 · Very relevant ≥ 52 · Related ≥ 36 — the top two also require genuine content overlap
- **Your sources** apply first: *Hide* removes a source (a hidden outlet isn't even fetched), *Prefer* ranks it as a top-tier publication, *Only preferred* shows nothing else, *English only* drops other languages and scripts
- **News feeds**: per-title topic feeds are about the title by definition; recent-coverage feeds keep only articles that name it in quotes, as a capitalised name, or next to its cast

Watch next

```
Wikidata: works sharing the 6 leading cast members, or the director / creator → order
by people shared, then fame (Wikipedia editions)
```

- With a TMDB key: TMDB's recommendations and similar titles, each of the top 10 checked for Netflix India (subscription offers only), mapped back to Wikidata so *Explore* can open them

4.5 Data sources and graceful degradation

Source	Providers	When something fails
Reddit	① Official OAuth API (client id) · ② public JSON · ③ official Atom feeds	Circuit breaker skips a blocked / rate-limited provider; one failing scope doesn't sink the others; feeds carry no counts and the UI says so
Wikidata	MediaWiki API (search, facts) · SPARQL query service (types, socials, Watch next)	Query service down → descriptions classify search results; socials simply missing; About still works

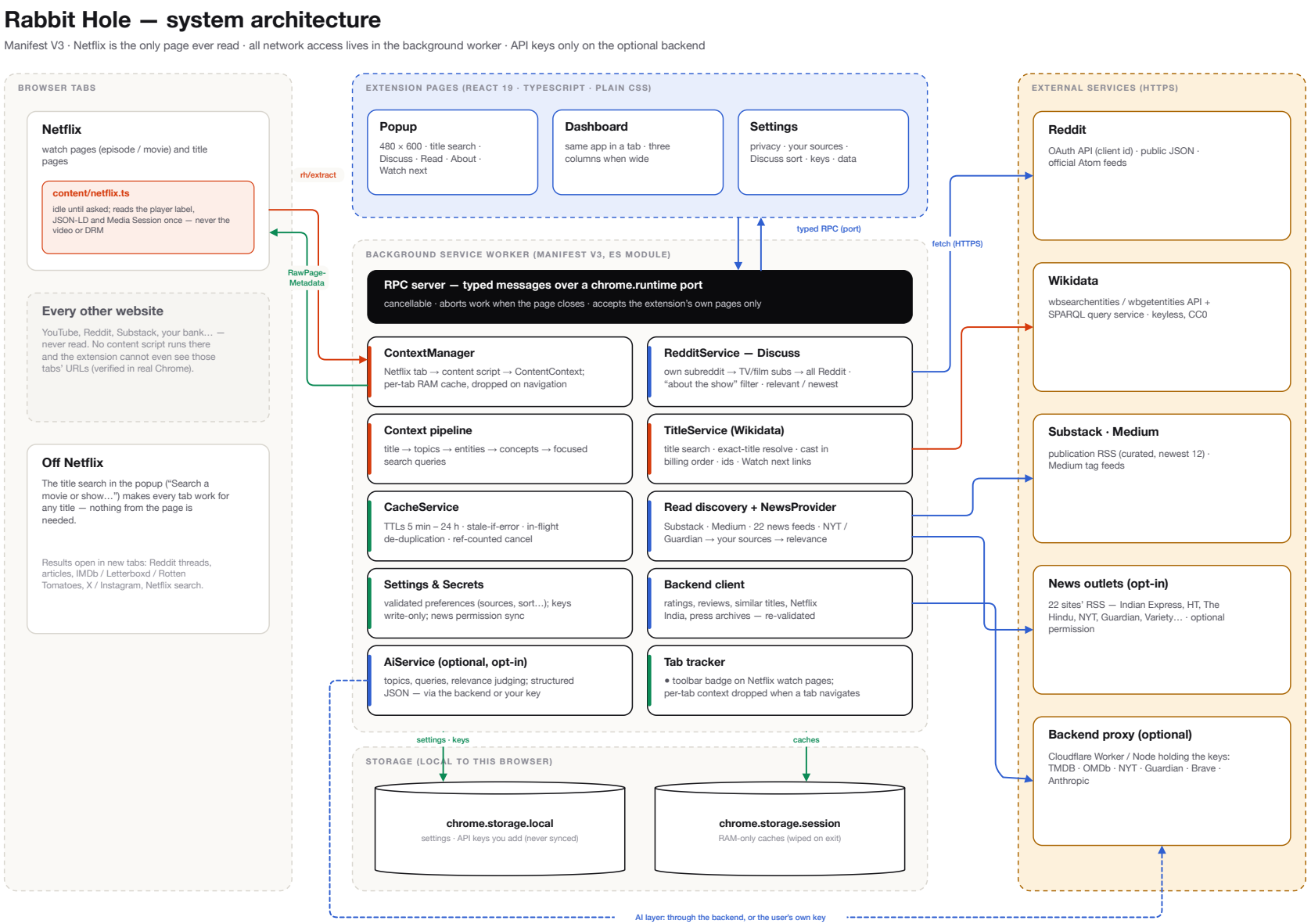
Source	Providers	When something fails
Substack	Backend / Brave key (`site:substack.com`), else 61 curated publications' RSS (33 custom domains opt-in)	Per-feed failures ignored; error only if all fail
Medium · news	Medium tag feeds · 22 outlets' RSS (opt-in permission)	An unknown topic page (404) means "no stories", not an error
Backend (optional)	TMDB · OMDb · NYT · Guardian · Brave · Anthropic — keys server-side	Each feature only with its key; tabs say what a backend would add; nothing is guessed

4.6 Caching and performance

Data	TTL	Behaviour
Reddit searches (Discuss)	5 min	Keyed by plan and sort; stale-if-error (flagged)
Read results · feeds	10 min · 30 min	Only the newest items of each feed are downloaded (streamed, cut off early)
Title search · title facts	1 h · 24 h	Wikidata changes slowly
Watch next · insights	24 h · 6 h	Watch next queries take 5–8 s cold, so they run only when the tab is opened
Page context	Until the tab navigates	Per-tab, RAM only (`storage.session`)
In-flight requests	—	Identical concurrent calls share one fetch; cancellation is reference-counted

Other measures: 250 ms debounced typeahead; Enter pressed before results arrive picks the first one when they do; stale requests cancelled on every change; skeleton paint before data; no polling; a 745 KB production build with a single small content script.

4.7 System architecture diagram

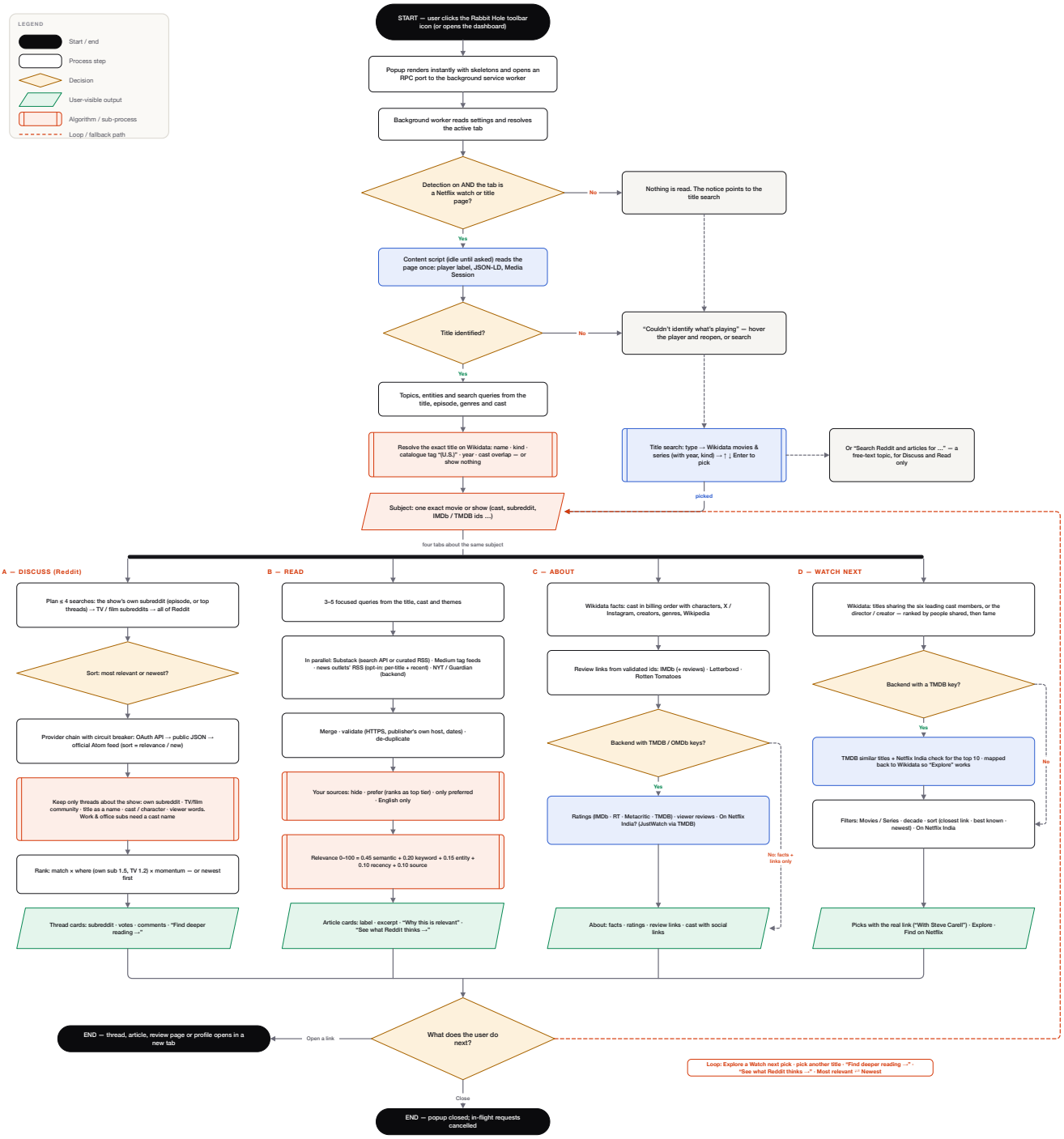


CHAPTER 5

Flowcharts

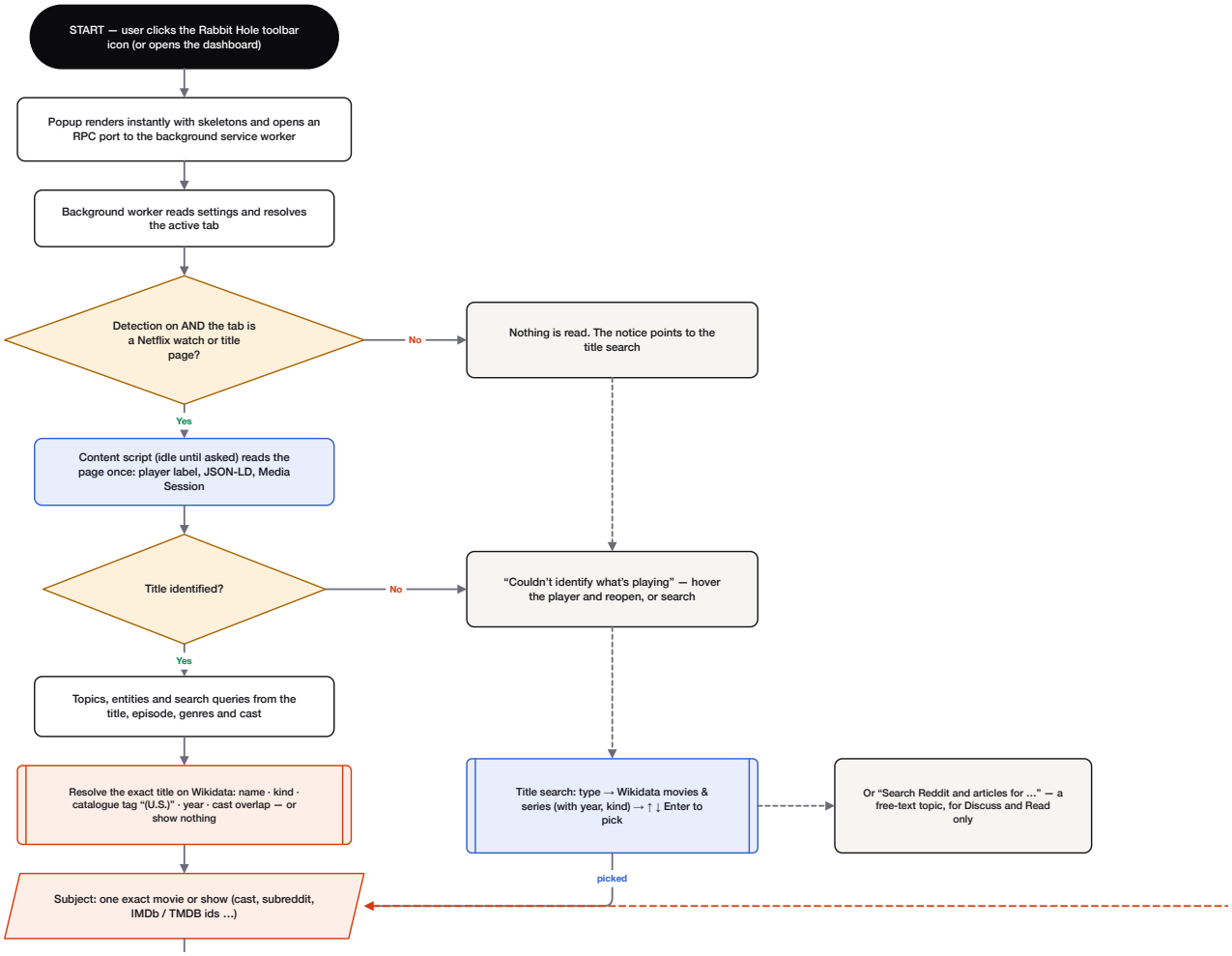
Three diagrams document the project; each exists as a vector SVG in `docs/flowcharts/` (plus PNG previews and Mermaid sources). The project flowchart (this section) follows standard flowchart symbols; the architecture diagram is in §4.7; the UI flow — built from the real screens and ready to import into Figma — closes this section.

5.1 Project flowchart — overview



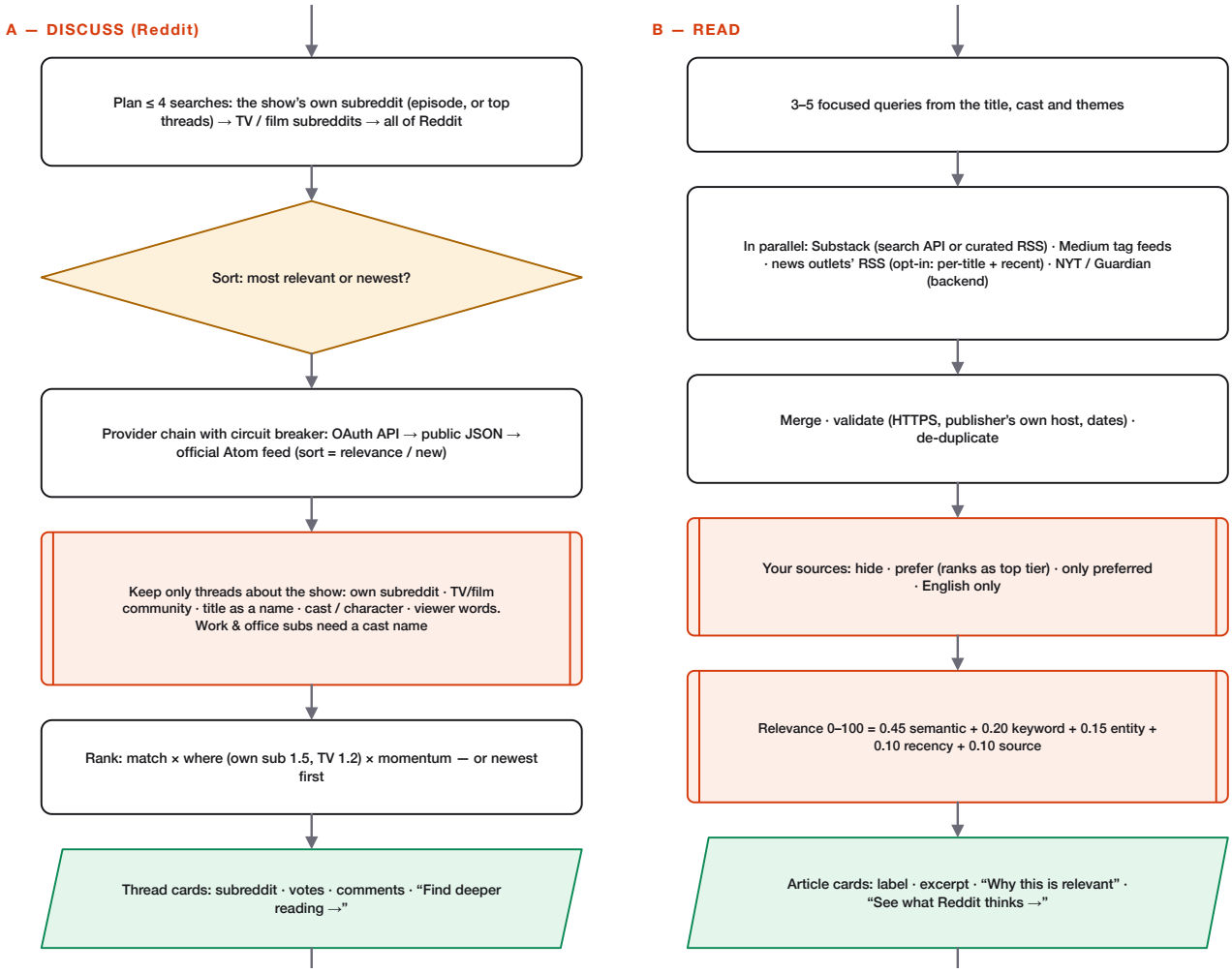
The next pages zoom into each part at a readable size.

5.2 Part 1 — Identifying the title



The popup paints skeletons immediately while the background resolves the active tab. A page is read **only** if detection is on *and* it is a Netflix watch or title page. The title is then resolved to one exact Wikidata item — or, when that can't be done with confidence, nothing is shown. Every fallback (any other site, hidden labels, detection off) leads to the title search, which rejoins at the same *subject* node; a free-text search feeds Discuss and Read only.

5.3 Part 2 — Discuss and Read



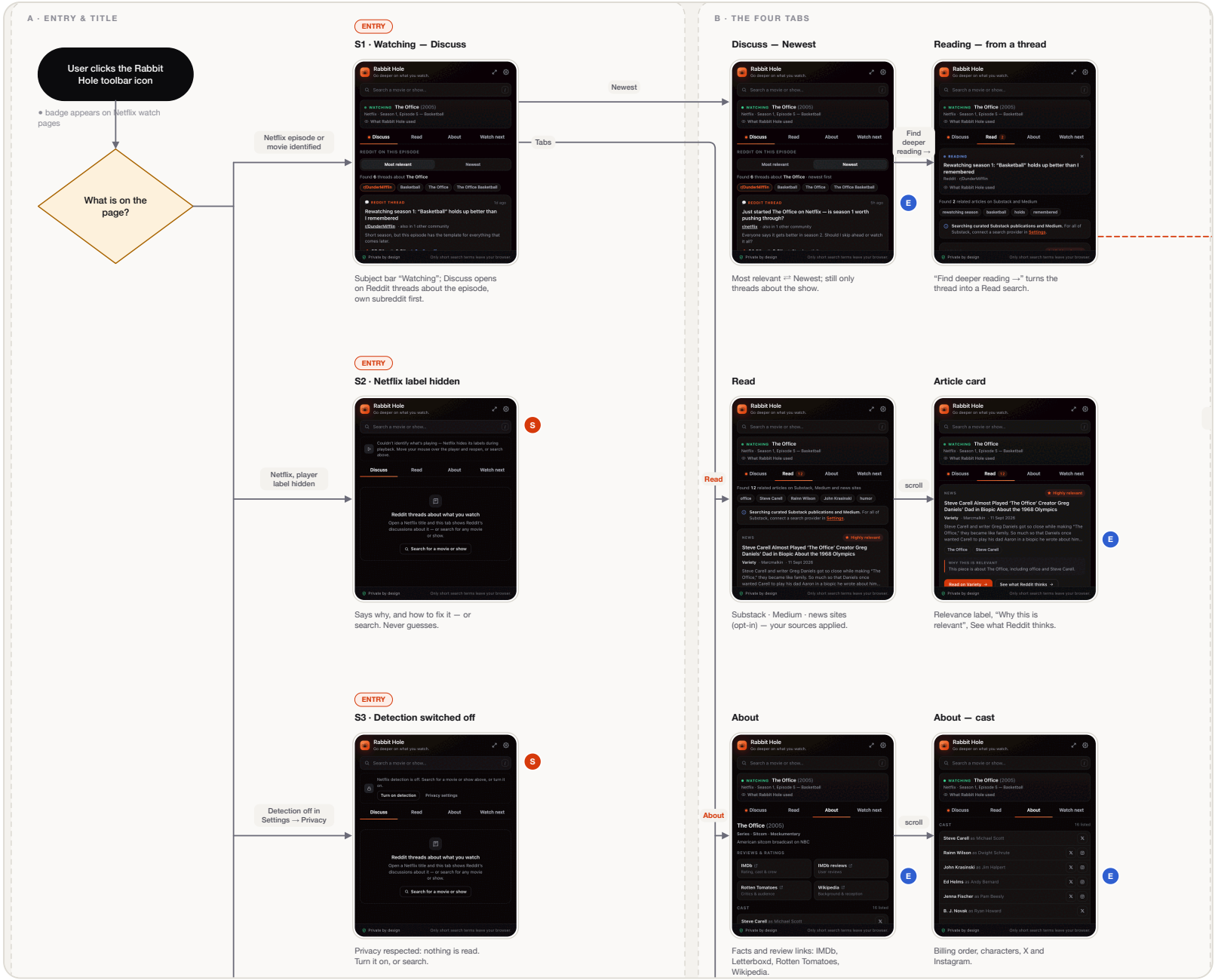
Discuss plans at most four Reddit searches — the show's own subreddit first — runs them through the provider chain in the chosen order (most relevant or newest) and keeps only threads that are clearly about the show. **Read** queries Substack, Medium, news outlets and (with the backend) the NYT and Guardian archives in parallel, applies the user's source preferences and the English-only filter, then scores every article 0–100 and explains it.

5.4 Part 3 — About, Watch next, and what the user does next

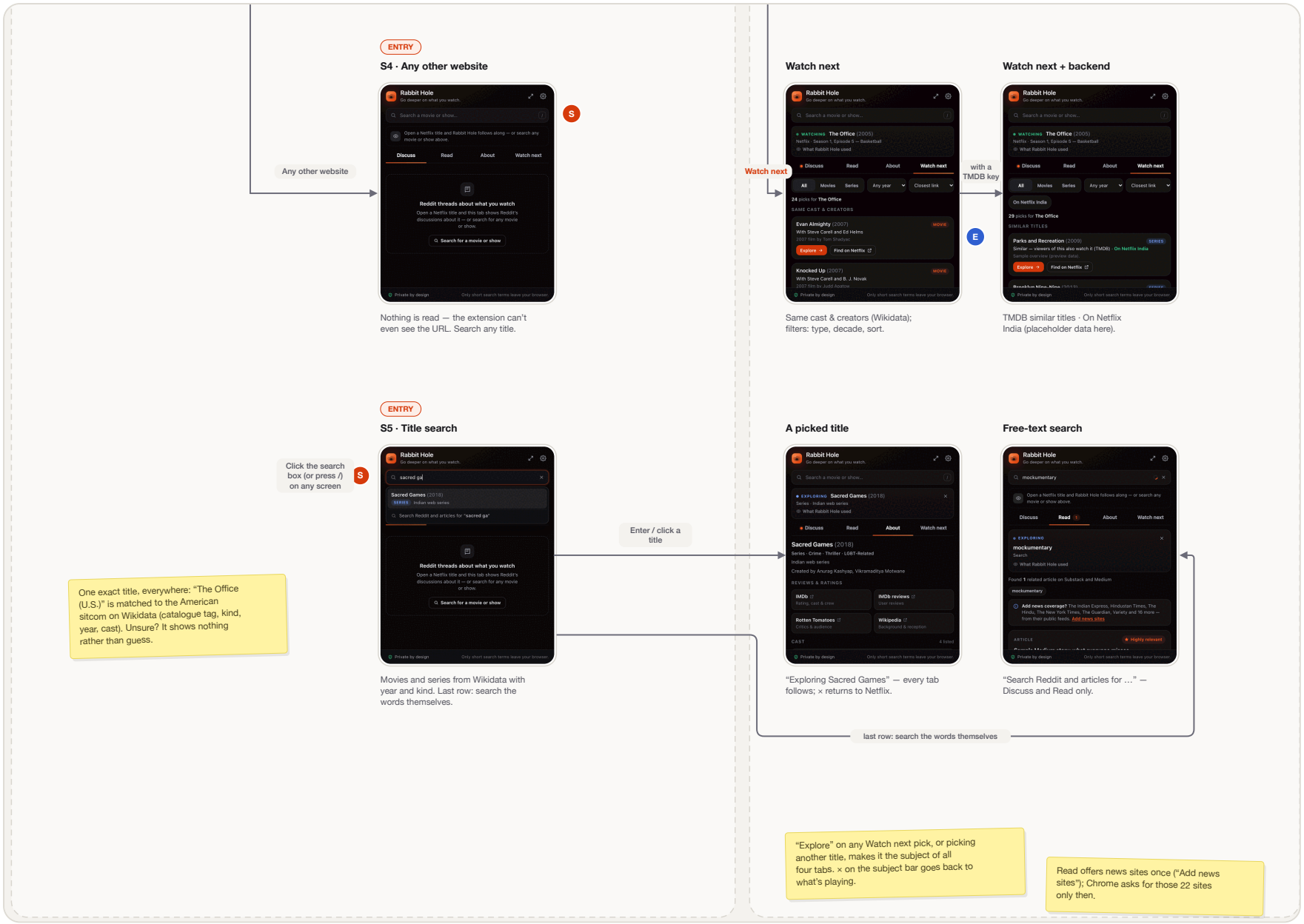


About and **Watch next** always work from Wikidata; with a backend holding TMDB / OMDb keys they add ratings, viewer reviews, similar titles and Netflix India availability. The dashed loop shows how Explore, picking another title, the bridges and the sort switch give the tabs a new subject or order.

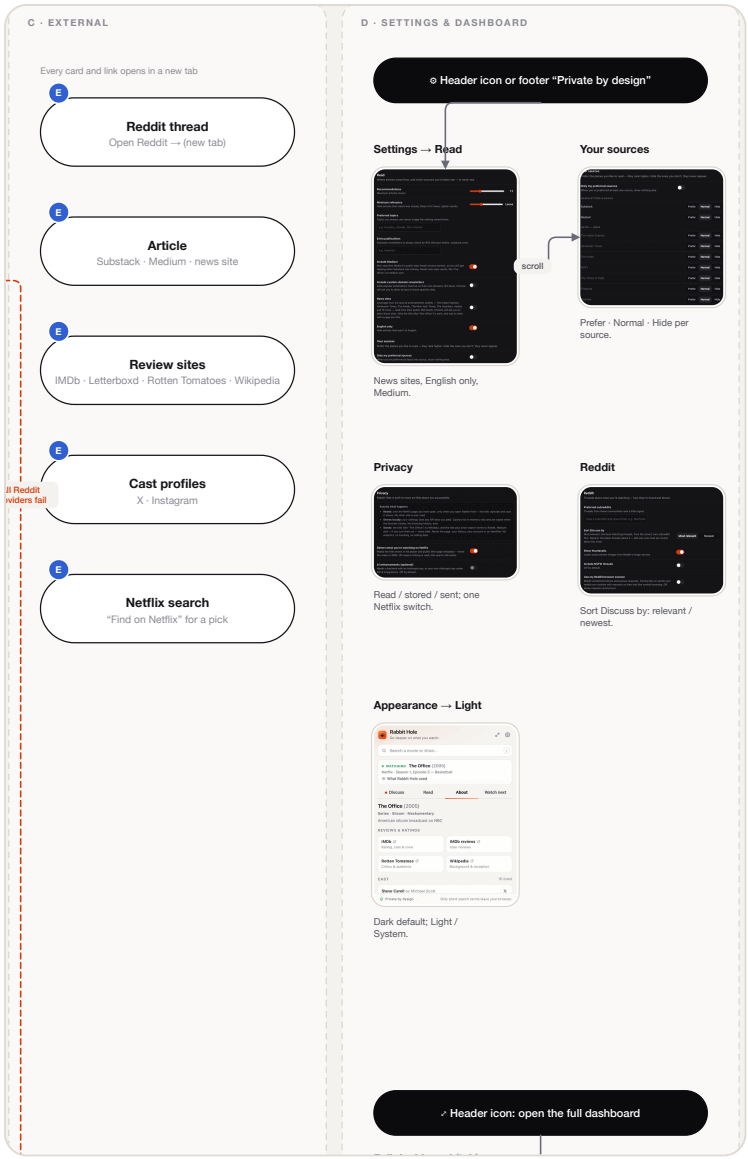
5.5 UI flow (Figma-ready) — A · entry & title, B · the four tabs · S1–S4, Discuss · Read · About · file: `docs/flowcharts/figma-ui-flow.svg`

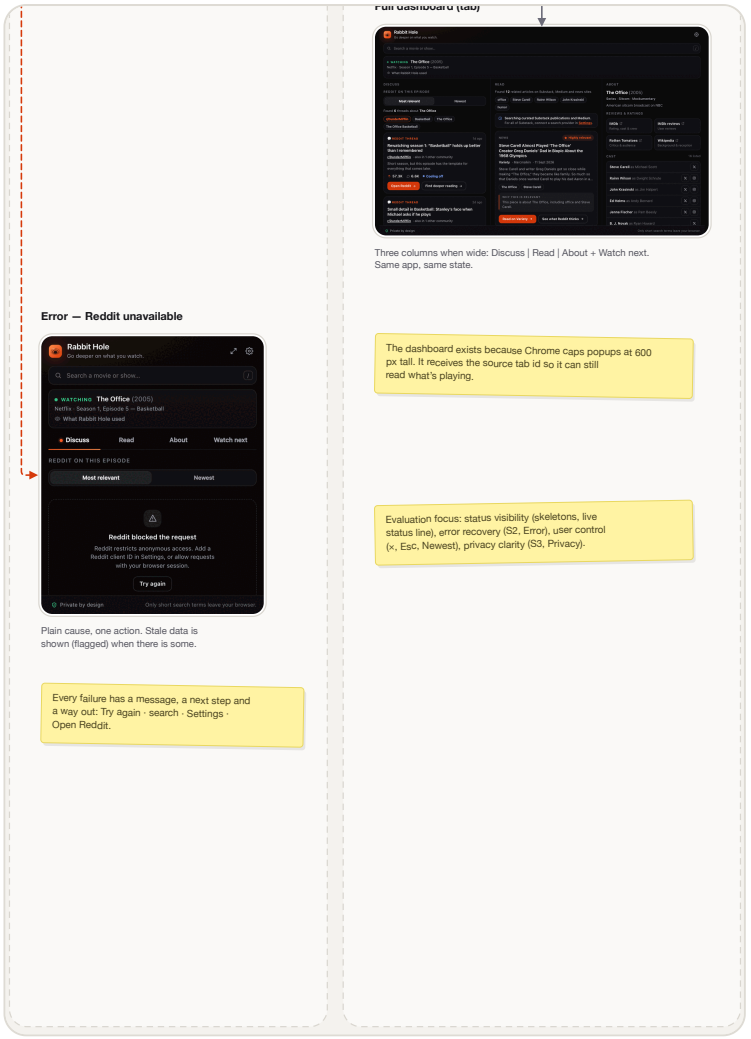


5.5 UI flow — S5 title search, Watch next, a picked title, free-text search · every tab follows the subject



5.5 UI flow — C · external links & errors, D · settings & dashboard





Using it in Figma: drag `figma-ui-flow.svg` onto a Figma or FigJam canvas (or File → Import). Frames, labels and arrows arrive as separate layers and the screenshots as image fills. `ui-flow.mmd` is a Mermaid version for native FigJam diagrams.

CHAPTER 6

Privacy and security

6.1 Privacy by design

Question	Answer
What is read?	Only the Netflix page the user has open, only when the extension is opened — the title, episode, genres and cast it shows. No other site is ever read: there is no content script elsewhere and the extension cannot even see other tabs' URLs (verified in real Chrome).
What is stored?	Settings (incl. source preferences) and any key the user adds (`storage.local`, never synced). Caches live in `storage.session` — RAM only. **No browsing history is kept.** Provider keys (TMDB, OMDb, NYT, Guardian) exist only on the backend.
What is sent?	The title to Wikidata; the title, episode name and short search terms to Reddit and Medium; with news on, the title as a slug ("the-office") to the seven outlets that have per-title pages; to the optional backend, the title's ids or name. Never page contents, URLs, history or identifiers.
Tracking / analytics?	None. No third-party scripts, fonts or telemetry. Nothing is sold.
User control	Netflix detection switch; "What Rabbit Hole used" on the subject bar; Your sources; news sites opt-in (and revocable); clear cache; delete everything.

6.2 Security controls

Threat	Control	Verified by
Malicious page text (XSS)	No HTML sinks: external text is rendered as React text nodes after control-character stripping and length caps	Source scan; sanitise tests
Script injection / remote code	CSP `script-src 'self'; object-src 'self'; base-uri 'none'`; no eval, inline or remote scripts	Release gate; real-Chrome run: 0 CSP violations
Malicious or compromised data source	Every response validated; article links must be HTTPS on the publisher's own host; About links built only from pattern-checked ids; backend output re-validated in the extension	parsers, news, insights tests
Key theft	Provider keys on the backend; user keys write-only (masked status), `storage.local` locked to trusted contexts; dev keys stripped from production builds	Release gate with sentinel keys; secrets tests
Hostile web page talking to the extension	Background accepts RPC only from the extension's own pages; the content script replies only to the extension and only to one message type	rpc tests (foreign sender refused)
Over-broad permissions	Two permissions (`storage`, `scripting`); hosts limited to Netflix, Reddit, Substack, Medium and Wikidata; 22 news and 33 newsletter origins only as *optional*, user-granted permissions	manifest tests; real-Chrome check
Backend abuse	Origin allow-list, per-client rate limit, size caps, per-instance caching (quota protection), no upstream error or key echo	backend, insights tests

CHAPTER 7

Testing and quality assurance

7.1 Automated tests — 263 passing in 21 files

Area	Tests	What is covered
Topic extraction & query generation	24	titles, catalogue tags, entities, Netflix themes, manual searches, AI-first queries
API failure handling (services)	21	Reddit OAuth → JSON → RSS chain, circuit breaker, stale data, degradation, empty results
API failure handling (HTTP)	19	401/403/404/429/5xx mapping, Retry-After, offline, timeout, cancel, size limits
Settings, secrets, sanitising	19	clamping, write-only keys, URL allow-lists, HTML → inert text
Relevance scoring	18	formula weights, 0–100 bounds, labels, content gating, explanations, diversity
Cache behaviour	17	TTLs, stale-if-error, in-flight de-duplication, ref-counted cancel, eviction, quota failure
Parsing & validation	17	Reddit JSON/Atom, RSS, search results, URL/date/author sanitising
Backend proxy	14	CORS, rate limit, validation, no key leakage, extension ⇄ backend contract
Manifest & build artefact	13	minimal permissions, Wikidata hosts, news sites optional, strict CSP
AI provider	12	request shape, prompt-injection fencing, structured output, refusals
Netflix metadata extraction	12	player label, movie vs episode, JSON-LD, Media Session, title-page enrichment, fallback
Backend: ratings, reviews, Netflix India, press	10	TMDB v3/v4 auth, IMDb lookup, watch providers, caching, NYT / Guardian, re-validation
Reddit ranking	10	momentum score, growth, cross-community counts, feed-only mode
Discuss filter and plan	9	own subreddit first, office-politics dropped, newest-first, sort setting
Platform detection	9	Netflix watch / title pages; every other site unknown; junk input
Title identification (Wikidata)	9	search, facts in billing order, “The Office (U.S.)” vs UK, cast tie-break, query service down
Medium	8	tag slugs, story detection, feed parsing, Substack + Medium together
RPC security	8	typed round-trips, cancellation, worker loss, foreign senders refused
Context manager	7	Netflix only, per-tab cache, privacy switches, undetected/idle states, badge
News & your sources	6	feed parsing, title-naming rule, opt-in, hidden outlets not fetched, prefer / only preferred, English only
Built service worker smoke	1	boots dist/background.js with no DOM and serves RPC

7.2 Release gate and real-browser end-to-end

Release gate (`npm run verify:release`): builds with sentinel secrets in the environment and proves they cannot appear in a production bundle; confirms the public client id and backend origin *are* compiled in; scans for `eval`, `new Function`, `document.write`, inline/remote scripts and source maps; checks content scripts are self-contained.

Real-Chrome end-to-end (`npm run test:e2e`, Chrome for Testing 155): loads the built `dist/`, serves fixture pages at the *real* Netflix, YouTube and Reddit URLs, and drives the real popup and Settings against the real service worker — **31 of 31 checks pass**. Highlights:

- Netflix episode extracted; ● badge on watch pages, cleared when detection is off
- YouTube, Reddit and an ordinary website are **not read**: no content script runs and the URL isn't even visible
- Four tabs, leading with Discuss; Discuss shows threads or its friendly blocked state
- Read: live Substack and Medium feeds through host permissions; the news offer appears
- About and Watch next from **live Wikidata**: Black Mirror's cast with social links, IMDb / RT links, "Cunk on Earth — From Charlie Brooker"
- Title search: "Sacred Games" → pick → every tab follows; free-text search still works
- Zero Content-Security-Policy violations; zero uncaught errors

7.3 Defects found and fixed during development

Reviewing real captures and live data surfaced genuine defects — each now has a fix and, where practical, a regression test.

#	Finding	Found by	Resolution
1	"The Office" on Reddit returned office-politics and Microsoft Office threads	Reviewer feedback (round 2)	Own subreddit first + "about the show" filter; live check: off-topic threads dropped
2	"The Office (U.S.)" could match the British series	Wikidata probing	Catalogue tag, kind, year and cast break ties; below a threshold, no match
3	Cast ordered by fame put cameos first (Serena Williams in Glass Onion)	Live Wikidata check	Billing order from statement order (wbgetentities)
4	A Portuguese Medium post slipped into English reading	Real-Chrome e2e	English-only filter, incl. Medium's English footer; test with the real text
5	Enter pressed before search results arrived did nothing	Real-Chrome e2e	Enter is remembered and picks the first result when it arrives
6	Discuss briefly flashed fallback results before the exact title was known	Screenshot automation	Discuss waits for the title lookup
7	Long footer / note lines caused sideways scrolling in the popup	Preview inspection	Notes wrap; overflow checked in the preview
8	Publication topic tags leaked into article scoring	Ranking review	Tags only select feeds; test asserts no effect on score
9	"Highly relevant" reachable from source quality + freshness alone	Ranking review	Top labels require content overlap
10	Reddit JSON blocked (403) and feeds rate-limited (429) from the build network	Live probing	Three-provider chain, circuit breaker, per-scope failures tolerated

CHAPTER 8

Usability evaluation plan

The product has been verified technically; whether people *understand and enjoy* it needs usability evidence. The accompanying **Heuristic & Usability Evaluation Form** (a fillable PDF, `docs/Rabbit-Hole-Heuristic-Evaluation-Form.pdf`) operationalises the plan below.

Method	Participants	Output	Primary measure
A · Heuristic evaluation (Nielsen's 10)	3–5 evaluators (UX / dev / peers)	Severity-rated problems per heuristic	Mean severity 0–4; # major problems
B · Cognitive walkthrough	2–3 evaluators	Step-level learnability failures	4 questions × steps for 7 tasks
C · Moderated think-aloud test	5 target users	Observer log per task	Completion, time, errors, SEQ (1–7)
D · SUS questionnaire	Same 5 users	Standardised usability score	SUS 0–100 (average ≈ 68)
E · Relevance & trust survey	Same 5 users	Precision of recommendations; trust/understanding of privacy	Precision@5; Likert 1–5; NPS
F · Accessibility audit (WCAG 2.2 AA)	1–2 evaluators + tools	Pass/fail checklist	# failed criteria
G · Error-state & performance matrix	1–2 evaluators	Expected vs observed behaviour	Pass rate; time to first paint / results
H · A/B experiment plan	≥ 20 users per variant (future)	Decisions on four open design questions	Click-through; time to first click

8.1 Design hypotheses to test

The design team enters the evaluation with explicit, falsifiable suspicions (listed in the form's facilitator notes and withheld from evaluators until they have finished independently, to avoid bias):

H-1 New users won't know what "Highly relevant" is based on; the explanation is only in a tooltip.

H-2 There is no first-run explanation of what the extension reads; privacy clarity depends on the footer and Settings.

H-3 Users may not realise the search box finds *titles*, or miss its last row ("Search Reddit and articles for …").

H-4 Four tabs in 480 px: "Watch next" may be overlooked, and users may not expect Discuss to be the first tab.

H-5 Netflix users whose labels are hidden may not see that the search is the way forward.

H-6 Without a backend, users may expect ratings and Netflix availability and not notice the note explaining why they're missing.

H-7 The *Your sources* list (24 rows) is long; users may not find *Only my preferred sources*.

H-8 "Newest" may surprise users when it shows fewer threads than "Most relevant".

8.2 Success criteria (proposed)

Metric	Target
Task completion (core tasks T1–T5)	≥ 80 % unassisted
T1 time (open on a Netflix title → open a related thread or article)	≤ 30 s median
SUS	≥ 75 (“good”)
Precision@5 of recommended articles (judged “relevant” or “partly”)	≥ 70 %
Heuristic evaluation: major (3) or catastrophic (4) problems	0 unresolved before release
Accessibility audit: failed WCAG 2.2 AA criteria	0 critical
“I understand what Rabbit Hole reads and sends” (Likert 1–5)	≥ 4.0 mean

CHAPTER 9

Limitations, risks and future work

9.1 Known limitations (stated plainly)

- **Reddit access is outside our control.** Reddit blocks the build machine's network in the browser; Discuss was verified with Reddit's live RSS from Node (r/DunderMifflin threads about the Basketball episode), captured feeds and mocked responses. Without a client id, feeds carry no vote counts.
- **The backend features are not yet verified live.** TMDB, OMDb, NYT and Guardian integrations are tested against their documented response formats; no keys were available (the Guardian's former public `test` key now returns 401).
- **Netflix markup changes without notice** and hides its labels during playback; detection was verified against fixtures, not a live subscription. Availability is checked for **India only**, and only for the top 10 similar titles.
- **Wikidata is community-edited.** Cast order, handles and ids are as good as its data; Indian titles often have no subreddit; a missing handle is simply not shown.
- **News coverage is uneven.** Indian outlets offer only recent section feeds (no per-title pages), so older Indian titles get little news; Western outlets with per-title feeds cover older titles too.
- **Watch next is slow the first time** (5–8 s per title on Wikidata's query service), then cached for 24 hours.
- **Popup height is capped at 600 px** by Chrome; the dashboard provides the roomy layout.
- **Usability is untested with users.** The evaluation pack exists to close that gap.

9.2 Future work

Priority	Item	Why
High	Deploy the backend with TMDB / OMDb / NYT / Guardian keys and verify each live	Closes the biggest verification gap
High	Run the evaluation pack with real users; iterate on severity ≥ 3 findings	Closes the usability evidence gap
High	First-run onboarding (what is read / sent; what the backend adds)	Addresses H-2, H-6
Medium	Reddit client-id onboarding; Chrome Web Store packaging and privacy policy	Reliable data + distribution
Medium	More regions for availability; more Indian sources with per-title pages	Reach
Medium	Embedding-based semantic scoring behind the backend	Replace the lexical approximation
Low	Other streaming services (Prime Video, Hotstar) via the extractor interface	One extractor + one URL rule each

Appendices

A · Permissions, explained

Permission	Why
`storage`	Settings (local), API keys you add (local, background-only) and RAM-only caches (session).
`scripting`	Injects the Netflix reader into a Netflix tab opened before the extension was installed.
`https://www.reddit.com/*`	Reddit data (feeds / JSON / official API) — Discuss
`https://oauth.reddit.com/*`	Reddit data (feeds / JSON / official API) — Discuss
`https://www.netflix.com/*`	Content script: “what is playing?”; ● badge
`https://*.substack.com/*`	Publication RSS feeds — Read
`https://medium.com/*`	Medium tag feeds — Read
`https://www.wikidata.org/*`	Title search, facts, cast, Watch next
`https://query.wikidata.org/*`	Title search, facts, cast, Watch next
`optional:` Brave, Anthropic	Requested at runtime only when the user adds that key
`optional:` 22 news-outlet origins	Requested once, when the user turns on news sites
`optional:` 33 custom-domain feeds	Requested only when “Include custom-domain newsletters” is turned on

Not requested: `tabs`, `activeTab`, `history`, `cookies`, `webRequest`, `<all_urls>`.

B · Technology stack

Layer	Technology	Version
Platform	Chrome Extension, Manifest V3 (ES-module service worker)	Chrome ≥ 116
Language	TypeScript (strict, noUncheckedIndexedAccess)	^7.0.2
UI	React · plain CSS design tokens (no CSS framework)	^19.3.0
Build	Vite (app) + per-script IIFE builds for content scripts	^8.3.3
Tests	Vitest · jsdom · puppeteer-core (e2e)	^5.0.3 · ^30.1.2 · ^25.12.0
Parsing	fast-xml-parser (RSS / Atom)	^5.11.2
AI (optional)	@anthropic-ai/sdk — structured JSON output	^0.131.0
Data	Wikidata (MediaWiki API + SPARQL) · Reddit · RSS (Substack, Medium, 22 outlets)	—
Backend (optional)	Web-standard handler on Cloudflare Workers / Node · TMDB · OMDb · NYT · Guardian	—

C · Repository layout

```
manifest.config.mjs    MV3 manifest generator (news origins from outlets.json)
src/
  background/          service worker: container, RPC handlers, contextManager, tabs
  content/             netflix.ts — the only content script
  popup/               App · Discuss · Read · About · Watch next panels · data hooks
  options/             Settings (incl. Your sources)
  components/          TitleSearch · SubjectBar · cards · tabs · skeletons · states
  services/            titleService (Wikidata) · reddit (+ aboutTitle) · news · substack
                       · context · ranking · cache · ai · settings · secrets · backend client
  utils/ types/        http, rpc, sanitize, text, errors · shared contracts
server/                optional backend: Brave · Anthropic · TMDB · OMDb · NYT · Guardian
tests/                 21 test files (+ e2e/)
docs/                  this report, evaluation form + results, flowcharts, screens
```

D · Glossary

Term	Meaning
Subject	The one movie or show every tab is about — identified on Netflix or picked in the title search
Context	The normalised description of what the user is looking at (`ContentContext`)
Wikidata / QID	The free knowledge base used to identify titles; each item has an id such as Q23831 (The Office, U.S.)
Provider	One interchangeable data source (e.g. Reddit OAuth, Reddit JSON, Reddit Atom)
Circuit breaker	Temporarily skipping a provider that just failed so it is not hammered
Stale-if-error	Serving expired cached data (flagged) when a refresh fails
Heuristic evaluation	Expert inspection of a UI against usability principles
SUS	System Usability Scale — 10-item questionnaire scored 0–100
SEQ	Single Ease Question — 1–7 rating after each task

E · References

1. Nielsen, J. (1994). *10 Usability Heuristics for User Interface Design.* Nielsen Norman Group. nngroup.com/articles/ten-usability-heuristics
2. Nielsen, J., & Molich, R. (1990). Heuristic evaluation of user interfaces. *Proc. ACM CHI '90*, 249–256.
3. Nielsen, J. (1994). Severity ratings for usability problems. Nielsen Norman Group.
4. Lewis, C., & Wharton, C. (1997). Cognitive walkthroughs. In *Handbook of Human-Computer Interaction* (2nd ed.). Elsevier.
5. Brooke, J. (1996). SUS: A "quick and dirty" usability scale. In *Usability Evaluation in Industry.* Taylor & Francis.
6. Sauro, J., & Lewis, J. R. (2016). *Quantifying the User Experience* (2nd ed.). Morgan Kaufmann.
7. W3C (2023). *Web Content Accessibility Guidelines (WCAG) 2.2.* w3.org/TR/WCAG22
8. Google. *Chrome Extensions — Manifest V3.* developer.chrome.com/docs/extensions
9. Reddit. *Data API documentation.* reddit.com/dev/api · Brave. *Search API.* brave.com/search/api
10. Wikimedia. *Wikidata Query Service* and *Wikibase API.* wikidata.org/wiki/Wikidata:Data_access
11. TMDB. *API documentation.* developer.themoviedb.org · OMDb API. omdbapi.com · NYT Article Search API. developer.nytimes.com · The Guardian Open Platform. open-platform.theguardian.com
12. Anthropic. *Claude API documentation.* docs.anthropic.com